Ch3

1) ~~Ratio~~ Each device stores $\frac{|A|}{4}$

Total bytes across all 64 devices = $64 \cdot \frac{|A|}{4} = 16|A|$

$\therefore$ Ans = 16 //

2) Time taken for $AllGather_X = \frac{\text{bytes (gathered output size)}}{\text{bidirectional ICI BW} \times \text{num-axes participating in collective opr}}$! TPU v4p

$= \frac{2BD/16 \cdot 4}{9.0e10 \cdot 2}$

$\simeq 23\mu s$ //

$AllGather_{XY} = \frac{2BD}{9.0e10 \cdot 2} = 23\mu s \times 2 = 46\mu s$ //

$AllReduce_Z = 2 \cdot AllGather_Z \Rightarrow AllReduce_Z([B_X, D_Y] \to [U_Z]) \to [B_X, D_Y]$

$\Rightarrow size = \frac{2BD}{XY}$

$\therefore T_{AllReduce_Z} = \frac{2BD}{XY}\left(\frac{1}{W \cdot 1}\right) \simeq 11.6\mu s$ //

3) $AllGather_X([B_X]) \to B \to Time = \frac{2B}{W \cdot 1} = \frac{2B}{W} = \frac{128.2}{9.0e10} \simeq 2.8ns$

But per-hop latency, estimating it to be 1μs per hop $\Rightarrow T_{hops} = 2\mu s$ //

$\therefore T = 2\mu s$ //

4)

Assume bf16	FLOPs	comms time
① AllGather + (Y[D,F])	$2BDF$	$\frac{2DF}{W}$
② AllReduce	$\frac{2BDF}{X}$	$\frac{4BF}{W}$ (AllReduce over $2BF$ bytes)

Let A = accelerator FLOPs/s

$$T_① = \max\left(\frac{2BDF}{A}, \frac{2DF}{W}\right)$$

$$T_② = \max\left(\frac{2BDF}{AX}, \frac{4BF}{W}\right)$$

① is compute bound when $\frac{2BDF}{A} > \frac{2DF}{W} \Rightarrow B > \frac{A}{W} = 2550$

② is compute bound when $\frac{2BDF}{AX} > \frac{4BF}{W} \Rightarrow X < \frac{2WD}{2A}$

$D = 8000$, $W = 180 \times 10^9$, $A = 459 \times 10^{12}$ for bf16 on TPU v5p,

Then $X < \frac{180 \times 10^9 (8000)}{2(459 \times 10^{12})} = \frac{90 \times 8000}{459 \times 10^3} = 1.56$

which is only possible when $X = 1$ ⇒ ② most likely, realistically, always comms bound ⇒ $T_② = \frac{4BF}{W}$

Since ① compute bound at $B > 2550$:

When $B > 2550$, $T_① = \frac{2BDF}{A}$, and $T_② = \frac{4BF}{W}$

Then $T_① > T_②$ if $\frac{2BDF}{A} > \frac{4BF}{W} \Rightarrow \frac{D}{A} > \frac{2}{W} \Rightarrow D > \frac{2A}{W} = 5100$

⇒ ② better for larger models at $B > 2550$

Then when $B < 2550$, $T_① = \frac{2DF}{W}$, $T_② = \frac{4BF}{W}$ (approx.)

$T_② > T_① \Rightarrow \frac{2DF}{W} > \frac{4BF}{W} \Rightarrow D > 2B \Rightarrow D > 5000$

Similarly, strategy ② better for larger models //

6) ~~A[I,J] · B[J,K]~~

X=Y=Z=4 and it's 4×4×4

Baseline: A[I, J] · B[J, K] → C[I, K]

Sharding along the non-contracting axes can avoid the expensive call of AllReduce.

∴ A[Ixyz, J] · B[J, K] then AllGather.

$FLOPs = \frac{2IJK}{XYZ}$, $comms = \frac{2IK}{3W}$

$T_{math} = \frac{2IJK}{XYZ \cdot A}$, $T_{comms} = \frac{2IK}{3W}$, $W = 9.0e10$, $A = 2.75e14$.

∴ for sharding Ixyz,

$T_{sharded} = \max\left(\frac{2IJK}{64A}, \frac{2IK}{3W}\right)$

$T_{baseline} = \frac{2IJK}{A}$

If sharded is compute bound, then $T_{shard} = \frac{2IJK}{64A} = \frac{IJK}{32A}$ which is 64x faster than $\frac{2IJK}{A}$

If comms bound, $T_{shard} = \frac{2IK}{3W}$ and $T_{shard} < T_{baseline} \Rightarrow$

$\frac{2IK}{3W} < \frac{2IJK}{A} \Rightarrow J > \frac{A}{3W} = \frac{2.75e14}{2.7e11} \approx 1000$

which is likely true ⇒ sharding is better than baseline.

A[I, J] · B[J, Kxyz] achieves equivalent results.

6) $A[I_x, J_Y] \cdot B[J_Y, K] \to C[I_x, K]$

Both contracting axes sharded → local partial sums

$C[I_x, K]\{U_Y\}$ ⇒ need to call AllReduce$_Y$

$T_{math} = \frac{2IJK}{XYC}$ for each local shard,

output shard size $= \frac{2IK}{X}$

$T_{comms} = 2 \times T_{AllGather} = 2 \times \frac{2IK}{XW} = \frac{4IK}{XW}$

- $A[I_X, J]B[J_X, K_Y]$ has A using X to shard output rows I but B using X to shard contracting dim J ⇒ conflict.
  ∴ First do an $AllGather_X\ B[J_X, K_Y] \to B[J, K_Y]$ then local matmul.
  $T_{math} = \frac{2IJK}{XYC}$, Gathered B size $= \frac{2JK}{Y}$
  $T_{comms} = \frac{2JK}{YW}$ Comm = $AllGather_X(B)$

- $A[I_X, J]\,B[J, K_Y]$
  No comms, $T_{math} = \frac{2IJK}{XYC}$

7) $In[B, D] \cdot Win[D, F] \cdot Wout[F, D]$
Size of matrices: $In: 2(128)(8192) = 2MB$
$Win = Wout = 2 \cdot (8192)(32768) \approx 540\,MB$
Need to shard matrices
$In[B, D_{XY}] \cdot Win[D, F_{XY}] \cdot Wout[F_{XY}, D] \to Out[B, D_{XY}]$
AllGather at the start:
$In[B, D_{XY}] \cdot Win[D, F_{XY}] \to In[B, D] \cdot Win[D, F_{XY}] \to H[B, F_{XY}]$
$AllGather_{XY}(In) \Rightarrow cost = \frac{2BD}{W \cdot 2} = \frac{BD}{W}$
$FLOPs = \frac{2BDF}{XY}$

$H[B, F_{xy}] \cdot W_{out}[F_{xy}, D] \rightarrow OUT[B, D]\{U_x\}$

Local Flops $= \frac{2BFD}{XY}$

- Reduce Scatter $\Rightarrow$ cost $= \frac{2BD}{2W} = \frac{BD}{W}$

Total Flops $= \frac{4BFD}{XY} \Rightarrow T_{math} = \frac{4BFD}{XYC} \simeq 174\mu s$

Total comms $= \frac{2BD}{W} \Rightarrow T_{comms} = \frac{2BD}{W} \simeq 23.3\mu s$

With overlap, $T = 174\mu s$, else w/o overlap $T = 197\mu s$

8)

```
@jax.pmap(axis_name="i")
def bench_all_gather(x):
    return jax.lax.all_gather(x, "i")
```

similar fnc as above

- all-reduce $\Rightarrow$ `jax.lax.psum(x, "i")`
- reduce-scatter $\Rightarrow$ `jax.lax.psum_scatter(x, "i", scatter_dimension=0, tiled=True)`
- all-to-all $\Rightarrow$ `return jax.lax.all_to_all(x, "i", split_axis=0, concat_axis=0, tiled=True)`

```
def main():
    y = fn(x)
    jax.block_until_ready(y)
```

(will run the actual code separately to report the timings)

9) Device r: $A_r = A[:, S_r]$ and $B_r = B[S_r, :]$

where S_r = set of indices assigned to device r:

e.g. $M=8, P=4 \Rightarrow S_0 = \{0,1\}, S_1 = \{2,3\} \ldots S_3 = \{6,7\}$

Computation: $C_r[n,k] = \sum_{m \in S_r} A[n,m]B[m,k] \Rightarrow C[N,K]\{Up\}$

AllReduce : $C[n,k] = \sum_{r=0}^{r-1} C_r[n,k]$

$$= \sum_{M=0}^{M-1} A[n,m]\, B[m,k]$$

b) In the last step, change AllReduce to ReduceScatter

c) $T_{AllGather} = \frac{2NM}{W} = T_1$ $\quad T_2 = \frac{2NK}{W}$ (matmul first)

$\frac{T_1}{T_2} = \frac{M}{K} \Rightarrow$ $M < K$: AllGather input cheaper

$M > K$: matmul first (ReduceScatter) cheaper

$M = K$: same cost

10) a) Each ~~dev~~ device will send a single shard of the matrix to its neighbor in each step of the algorithm, each one of size $\frac{N}{D} \times N$, and need to do it $D-1$ times $\Rightarrow \frac{N^2}{D}(D-1) = \frac{N^2(D-1)}{D}$

b) The difference is that in AllToAll, not every shard needs to go to every other device, eg. if D_0 contains $[A_0, B_0, C_0, D_0]$, then $A \to D_0$, $B \to D_1$, $C \to D_2$, $D \to D_3$, each of these are $\frac{N}{D} \times \frac{N}{D}$ in size, but total "movement" $= D-1 + D-2 + \dots + 1 + 0 = \frac{D(D-1)}{2}$

$\therefore \frac{N^2}{D^2} \times \frac{D(D-1)}{2} = \frac{N^2(D-1)}{2D}$

($\frac{N^2}{D^2}$ because $[A|B|C|D]$ holds $\frac{N^2}{D}$ in size, then each of A, B, C or D holds $\frac{N^2}{D^2}$)

c) $\frac{(b)}{(a)} = \frac{1}{2} \Rightarrow$ AllToAll is half the cost of AG/RS in unidirectional ring topology

Intuitively, for AG/RS, each devices' full shard must travel to every other device, but for AllToAll, each partial block has one destination

$\Rightarrow 1+2+\dots+D-1 \simeq \frac{D^2}{2}$ vs $D+D+\dots+D \simeq D^2$

$\Rightarrow \frac{1}{2}$ ratio //

d) The worst case hop is now halved ⇒ each chord only travels about half the ring since both directions operate simultaneously ⇒ total time halves

e) $1 + 2 + \dots + \frac{D-1}{2} \approx \frac{D^2}{8}$, saves a factor of 4

f) In unidirectional, AllToAll is 2x faster than AG ⇒ $ATA_{uni} = 2x\ AG_{uni}$

In bidirectional, $ATA_{Bi} = 4x\ ATA_{uni}$

$AG_{Bi} = 2x\ AG_{uni}$

$\therefore ATA_{Bi} = 4x\ (2x)\ AG_{uni} = 8x\ AG_{uni} = 4x\ AG_{Bi}$ //